

SAS to PySpark Migration via AI Agents

```
$ agent run sas-to-pyspark
ticket: migrate amortization logic
context: README, AGENTS.md, trace.json
rule: plan before code
gate: tests + diff + review
```

Hook: "Would you let Copilot commit directly to production?"

No. Let AI implement. Keep humans accountable.

Operating model: manage the agent like a junior developer
Human judgment stays in the loop. The agent accelerates implementation.

01 **Human defines**
Requirements, architecture, risk decisions

02 **AI implements**
Plan first, then code, tests, refactors

03 **Automation validates**
Unit tests, lint, type checks, security scans

04 **Human approves**
Review the PR, own the final decision

Give good tickets
Which request would you hand to a new team member?

Bad: "Rewrite the entire platform."

Good: small, scoped engineering tickets

- Add OAuth refresh token support
- Refactor login middleware
- Add integration tests
- Update API documentation

Onboard the agent
AI has the same problem a new hire has: no context, no confidence.

README

Architecture diagram

Coding standards

AGENTS.md

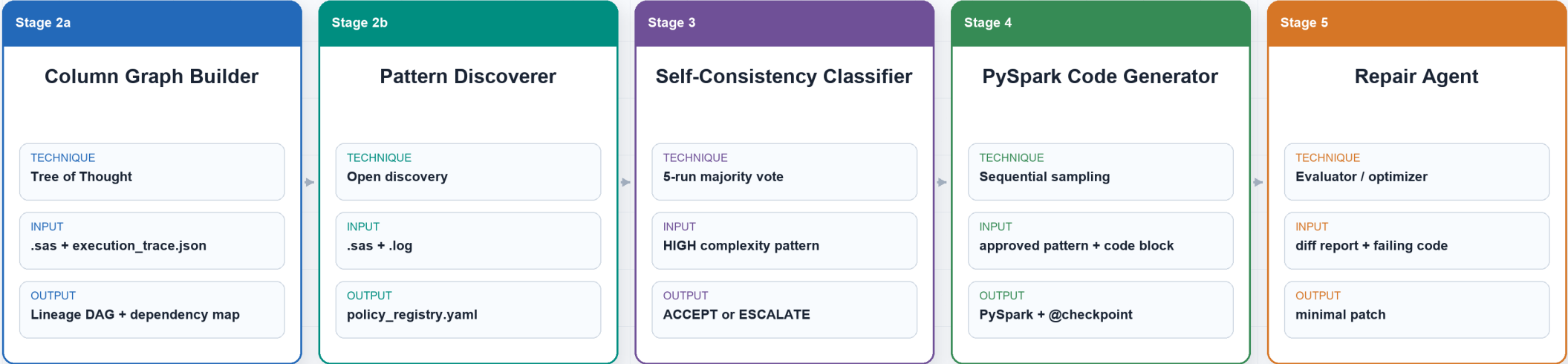
Build commands

Test commands

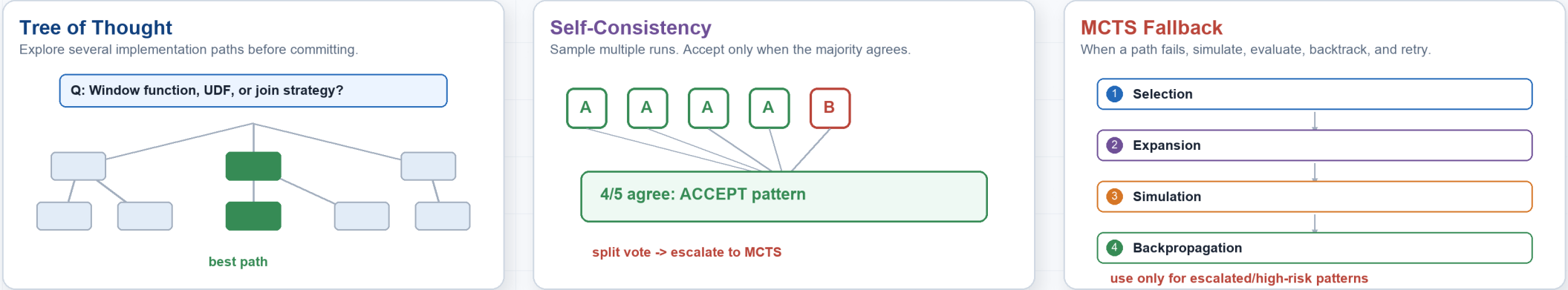
Document conventions, hidden build steps, review expectations, and architecture before asking the agent to write code.

5-stage prompt engineering pipeline

The agent must understand the SAS program, classify risk, generate candidates, and repair against evidence.



Reasoning patterns used inside the pipeline



Safety net: trust through validation
"Trust" is not a process. Verification is.

AI code

@checkpoint

Diff engine

Repair

Human review

Measure it
Treat AI agents like any other engineering system.

PR acceptance
usefulness

Test pass rate
correctness

Time saved
productivity

Defect rate
quality

The winning pattern is not AI replacing developers.
The winning pattern is developers who learn how to manage AI agents.

Let AI do the typing. Humans must still do the thinking.

SAS migration | Prompt engineering | Human review | Automated validation